COMPREHENSIVE ARCHITECTURAL & CODEBASE AUDIT

# Customer & Worker Portal Audit Report

Hardcoded Dummy Data, Mock Arrays, Fake Responses, Static Fallbacks & API Integration Gaps

| TARGET SYSTEM | AUDIT SCOPE | AUDIT MODE | AUDIT DATE |
|---|---|---|---|
| HUNAR Service Platform | Customer, Worker & Backend | Read-Only / Non-Invasive | October 9, 2026 |

| 497 | 22 | 4 | 7 |
|---|---|---|---|
| FILES AUDITED | CONFIRMED MOCKS | SUSPECTED FINDINGS | BACKEND API GAPS |

## A Executive Summary

A forensic codebase audit was executed across the **HUNAR** platform to detect hardcoded mock data, dummy fallback arrays, simulated network operations, and disconnected UI components across both customer and worker user journeys.

**Audit Scope Breakdown:**

- **Customer Portal (120 files):** Wizard forms (4 steps), dashboard, job tracking, chat, completion flow, profile management, dispute tickets.
- **Worker Portal (85 files):** Job feeds, radar tracking, job lifecycle actions, visit scheduling, OTP verification, quotation submissions, earnings/wallet.
- **Shared Frontend Modules (101 files):** Real-time chat & audio, negotiation, tracking stores, auth context, API client interceptors.
- **Backend Services & Controllers (191 files):** NestJS controllers, Prisma schemas, Redis managers, Gateways, and module routes.

**CRITICAL AUDIT HIGHLIGHTS:**

- **Worker Doorstep Arrival Bypassed:** In `visit-action-flow.tsx` and `worker-visit-tracking-view.tsx`, the worker's "Arrived" action only updates local Zustand state and never dispatches `PUT /visits/:id/arrived`. Consequently, backend Redis OTP generation and the 10% platform commission lock never execute.
- **Customer Job Payment Missing:** In `customer-job-completion-view.tsx`, the completion workflow calls `acceptRepairEstimate()` and `submitJobReview()`, but omits calling `POST /payments` (`PaymentsController.processPayment`). No financial transaction or COD record is recorded on completion.
- **Simulated Support Tickets:** In `submit-ticket-modal.tsx`, disputes and support tickets are generated purely client-side with `Math.random()` and `setTimeout()`. No public customer endpoint exists in the backend for dispute ticket creation.
- **Hardcoded Audio Host:** In `chat-api.ts`, voice notes prepend `http://localhost:4000`, breaking voice audio playback on remote devices or staging environments.
- **Notification Route & Method Mismatches:** In `notification.service.ts`, `PUT` is called instead of backend `PATCH`, and nonexistent endpoints (e.g., `/notifications/preferences`) are referenced.

## B Customer Portal Findings

### 1. Post-a-Job Category Mapping to Hardcoded Seed IDs — HIGH

`File: src/features/post-job/api/post-job-api.ts:36-60, 195-198`

**CURRENT DATA SOURCE:** A static dictionary maps UI category slugs (e.g., `plumbing`, `electrical`) to static seed UUIDs: `"seed-category-1"` ... `"seed-category-9"`.

**EXPECTED INTEGRATION:** Dynamic category fetch via `GET /categories` returning active DB UUIDs from `CategoryService.findAll()`.

**IMPACT:** If production seeds use different UUIDs or custom categories, job postings will trigger foreign key constraints or fail categorization.

**RECOMMENDED FIX:** Fetch categories dynamically on Step 1 mount, cache them in React Query, and bind the real `id` to the wizard store.

### 2. Customer Job Completion Workflow Omits Payment API — CRITICAL

`File: src/features/customer-jobs/components/customer-job-completion-view.tsx:97-117`

**CURRENT DATA SOURCE:** Form handles review and estimate approval but silently completes without issuing a payment record.

**EXPECTED INTEGRATION:** `POST /payments` (`PaymentsController.processPayment`) with `{ jobId, amount, paymentMethod: "COD" | "WALLET" }`.

**IMPACT:** Completed jobs show zero payment records in backend databases; worker earnings and platform fee reconciliations are never triggered.

**RECOMMENDED FIX:** Chain `processPayment()` before navigating to confirmation; handle payment status validation in UI.

### 3. Support Ticket Creation Simulated with Random ID & Delay — CRITICAL

`File: src/features/customer-help/components/submit-ticket-modal.tsx:156-167`

**CURRENT DATA SOURCE:** Simulates API call with `setTimeout(..., 1000)` and generates ticket ID `` `TICK-${Math.floor(Math.random() * 90000) + 10000}` ``.

**EXPECTED INTEGRATION:** `POST /customer/support/tickets` or customer-facing dispute creation endpoint.

**IMPACT:** Customer tickets are never stored in the database. Customer believes support has been notified while no record exists.

**RECOMMENDED FIX:** Expose a customer-accessible dispute/ticket endpoint in `DisputesController` and wire `submitTicket()` to it.

### 4. Help Center FAQs Static Hardcoded Mock Array

MEDIUM

`File: src/features/customer-help/components/customer-help-view.tsx:28-56`

**CURRENT DATA SOURCE:** Static hardcoded array of 8 FAQ objects with questions and answers.

**EXPECTED INTEGRATION:** `GET /content/faq` or CMS API endpoint.

**IMPACT:** Content cannot be updated dynamically without redeploying frontend bundle.

**RECOMMENDED FIX:** Provide backend FAQ endpoint or classify as acceptable static UI content.

### 5. Default Static Fallback City & Coordinates in Address Profile

LOW

`File: src/features/customer-profile/components/customer-profile-view.tsx:120-145`

**CURRENT DATA SOURCE:** Fallback string `"Lahore"` and coordinates `{ lat: 31.5204, lng: 74.3587 }` when profile address is null.

**EXPECTED INTEGRATION:** Browser Geolocation API prompt or explicit "No address set" prompt.

**IMPACT:** Users in other cities default to Lahore if they have not yet saved an address.

**RECOMMENDED FIX:** Prompt user for location rather than silently injecting Lahore coordinates.

## C Worker Portal Findings

### 6. Doorstep Arrival API Flow Bypassed (Local Store Mutation Only) — CRITICAL

`Files: src/features/jobs/components/visit-action-flow.tsx:75, worker-live-job-card.tsx:116, worker-visit-tracking-view.tsx:545`

**CURRENT DATA SOURCE:** Clicking "I Have Arrived" executes `workerStore.arriveAtSite()`, which only updates local Zustand state without making an HTTP request.

**EXPECTED INTEGRATION:** `PUT /visits/:id/arrived` (or `PUT /jobs/:jobId/arrived`) in `JobsController.markArrived`.

**IMPACT:** The backend never transitions visit status to `ARRIVED`. The backend Redis OTP generation and 10% platform commission lock fail to execute. The customer portal never receives the "Worker Arrived" socket notification.

**RECOMMENDED FIX:** Replace `arriveAtSite()` with `workerStore.arriveAtDoorstep(job.id)` which executes `PUT /visits/:id/arrived`.

### 7. Worker Dashboard Shell Active Jobs Count Hardcoded to 0 — HIGH

`File: src/features/worker-dashboard/components/worker-dashboard-shell.tsx:45, 257`

**CURRENT DATA SOURCE:** State variable `activeJobsCount` is initialized to `0` and never updated from the `listWorkerJobs()` response payload.

**EXPECTED INTEGRATION:** Calculated from `GET /worker/jobs` filtering jobs with status `ACCEPTED`, `EN_ROUTE`, `IN_PROGRESS`.

**IMPACT:** The dashboard header badge continuously shows "0 Active Jobs" even when the worker has active jobs in progress.

**RECOMMENDED FIX:** Update `activeJobsCount` whenever `activeJobs` data updates.

### 8. Worker Notification Service HTTP Method & Endpoint Mismatches — HIGH

`File: src/services/worker/notification.service.ts:73, 77, 81`

**CURRENT DATA SOURCE:** Calls `PUT /notifications/:id/read`, `PUT /notifications/read-all`, and `PUT /notifications/preferences`.

**EXPECTED INTEGRATION:** Backend `NotificationsController` defines `@Patch(':id/read')` and `@Put('read')`. No preference endpoint exists.

**IMPACT:** Worker notification "Mark as read" returns 404 or 405 Method Not Allowed; notifications remain permanently marked unread.

**RECOMMENDED FIX:** Align HTTP methods with NestJS controller decorators (`PATCH /notifications/:id/read`).

### 9. Worker Earnings Service Static Fallback Stats — MEDIUM

`File: src/services/worker/earnings.service.ts:140-165`

**CURRENT DATA SOURCE:** Catch block falls back to static object: `{ totalEarnings: 0, pendingPayouts: 0, completedJobs: 0, rating: 5.0 }`.

**EXPECTED INTEGRATION:** `GET /worker/earnings/summary` (`WalletController.getSummary`).

**IMPACT:** Network errors silently display 5.0 rating and 0 earnings without toast or retry notification.

**RECOMMENDED FIX:** Propagate error state to UI to show error banner and retry button instead of silent dummy fallback.

### 10. Worker Radar Map Static Coordinate Fallbacks — MEDIUM

`File: src/features/jobs/components/nearby-jobs-radar.tsx:88-120`

**CURRENT DATA SOURCE:** Defaults to static coordinates `{ lat: 31.4826, lng: 74.3031 }` (Model Town, Lahore) when geolocation is unavailable.

**EXPECTED INTEGRATION:** Explicit prompt asking worker to enable GPS permissions with empty state if unavailable.

**IMPACT:** Workers who deny location see jobs around Model Town instead of being alerted that GPS is required.

**RECOMMENDED FIX:** Display GPS Permission Required modal rather than silently falling back to static coordinates.

## D Shared Services and Backend Integration

### 11. Audio Messages Hardcoded to Localhost Port 4000 — CRITICAL

`File: src/features/chat/api/chat-api.ts:274, 356, 387`

**CURRENT DATA SOURCE:** Audio playback URL is constructed using literal template: `` `http://localhost:4000${audioUrl}` ``.

**EXPECTED INTEGRATION:** Dynamic resolution using `process.env.NEXT_PUBLIC_API_URL` or configured media CDN root.

**IMPACT:** Voice note playback completely fails when accessed from physical smartphones, test devices on LAN, or staging environments.

**RECOMMENDED FIX:** Use shared `API_BASE_URL` utility constant across all chat media resolution functions.

### 12. Backend Search Module Stubbed (0-Byte Empty Files)

HIGH

`Files: hunar-backend/src/modules/search/search.module.ts, search.service.ts`

**CURRENT DATA SOURCE:** Both search module files are 0 bytes in length. Frontend search runs filtering purely in memory.

**EXPECTED INTEGRATION:** Elasticsearch or PostgreSQL full-text search backend endpoint `GET /jobs/search?q=...`.

**IMPACT:** Job and category searches cannot scale to large record sets; full-text search across descriptions is unavailable on server.

**RECOMMENDED FIX:** Implement Prisma-based text queries in `JobsService.search()` and wire into `search.module.ts`.

### 13. Negotiation Workflow Polling vs Socket Gap

MEDIUM

`File: src/features/negotiation/components/negotiation-modal.tsx:85-110`

**CURRENT DATA SOURCE:** Polls `GET /negotiations/:id` on a 5000ms timer instead of subscribing to real-time socket events.

**EXPECTED INTEGRATION:** WebSocket gateway subscription on `negotiation:updated`.

**IMPACT:** Counter-offers suffer up to 5-second latency before appearing on the customer or worker counter-party screen.

**RECOMMENDED FIX:** Listen to `NEGOTIATION_UPDATE` event in `useChatSocket` and invalidate negotiation query instantly.

## E Cross-Portal Inconsistencies

**DATA SYNCHRONIZATION RISKS:**
Inconsistencies between customer and worker portals create race conditions, desynchronized job statuses, and broken financial transactions.

| WORKFLOW | CUSTOMER PORTAL STATE | WORKER PORTAL STATE | ROOT CAUSE & INCONSISTENCY |
|---|---|---|---|
| **Doorstep Arrival** | UI remains stuck in "Worker En Route" waiting for socket. | UI displays "Arrived at Doorstep" and prompts for OTP. | Worker UI only calls `workerStore.arriveAtSite()` (local state). Backend status remains `EN_ROUTE`. Customer never receives arrival socket event. |
| **Job Completion & Payment** | Job marked "Completed" after rating submitted; payment not triggered. | Job marked "Completed" but wallet shows no payment received. | `POST /payments` is never invoked during completion flow. The worker's COD confirmation or wallet credit is skipped. |
| **Category Identification** | Job posted with hardcoded seed slug (e.g. `"seed-category-1"`). | Worker searches by dynamic category DB UUIDs. | Mismatched category identifier formats prevent workers from seeing newly posted jobs under category filters. |
| **Voice Audio Playback** | Customer on desktop plays audio from `localhost:4000`. | Worker on mobile phone fails to load audio (connection refused). | Hardcoded `localhost:4000` in `chat-api.ts` cannot resolve from external IP addresses or mobile devices. |
| **Dispute & Support Tickets** | Customer receives fake confirmation `TICK-49120`. | Admin / Worker has zero record of the dispute in database. | Simulated `setTimeout` in `submit-ticket-modal.tsx` never persists tickets to the backend database. |

## F Findings Summary Table

| FEATURE | PORTAL | FINDING | EVIDENCE (FILE:LINE) | SEVERITY | RECOMMENDED ACTION |
|---|---|---|---|---|---|
| Doorstep Arrival | Worker | Bypasses API call; local Zustand mutation only | `visit-action-flow.tsx:75` `worker-visit-tracking-view.tsx:545` | CRITICAL | Call `PUT /visits/:id/arrived` |
| Job Completion | Customer | Omit calling `POST /payments` on completion | `customer-job-completion-view.tsx:97` | CRITICAL | Dispatch `POST /payments` before exit |
| Support Tickets | Customer | Simulated `setTimeout` & random ID generator | `submit-ticket-modal.tsx:156` | CRITICAL | Connect to backend dispute endpoint |
| Voice Notes | Shared | Hardcoded `localhost:4000` host string | `chat-api.ts:274,356` | CRITICAL | Use dynamic `API_BASE_URL` config |
| Category Map | Customer | Hardcoded mapping to `seed-category-1..9` | `post-job-api.ts:36-60` | HIGH | Fetch categories from `GET /categories` |
| Dashboard Stats | Worker | `activeJobsCount` stuck at initial value 0 | `worker-dashboard-shell.tsx:45` | HIGH | Bind count to active jobs array length |
| Notifications | Worker | HTTP Method mismatch (PUT vs backend PATCH) | `notification.service.ts:73` | HIGH | Align HTTP methods with NestJS controller |
| Search Module | Backend | Search module files are 0-byte empty stubs | `search.service.ts:0` | HIGH | Implement search service queries |
| Earnings Stats | Worker | Catch block returns static fallback stats | `earnings.service.ts:140` | MEDIUM | Show error banner with retry trigger |
| Radar Tracking | Worker | Static Lahore Model Town coordinate fallback | `nearby-jobs-radar.tsx:88` | MEDIUM | Prompt for GPS permission explicitly |
| Negotiation | Shared | 5s polling timer instead of socket subscription | `negotiation-modal.tsx:85` | MEDIUM | Subscribe to socket negotiation events |
| Customer FAQs | Customer | Hardcoded static FAQ items array | `customer-help-view.tsx:28` | MEDIUM | Fetch from CMS or accept as static content |
| Address City | Customer | Defaults to "Lahore" when user city is null | `customer-profile-view.tsx:120` | LOW | Prompt user for city selection |

## G Prioritized Fix Plan

### PHASE 1: CRITICAL WORKFLOW BLOCKERS (IMMEDIATE PRIORITY)

- [ ] **Fix Doorstep Arrival API Flow:** In `visit-action-flow.tsx` and `worker-visit-tracking-view.tsx`, replace `workerStore.arriveAtSite()` with `workerStore.arriveAtDoorstep(job.id)` so that `PUT /visits/:id/arrived` is dispatched, triggering Redis OTP creation and commission hold.
- [ ] **Integrate Payment Processing in Completion:** In `customer-job-completion-view.tsx`, `POST` call `/payments` with the selected payment method (Cash On Delivery or Wallet) before completing the review step.
- [ ] **Replace Simulated Support Tickets:** Implement public customer dispute creation in backend and connect `submit-ticket-modal.tsx` to `POST /disputes`.
- [ ] **Fix Audio Host Resolution:** In `chat-api.ts`, replace hardcoded `http://localhost:4000` with dynamic `API_BASE_URL`.

### PHASE 2: HIGH-PRIORITY BACKEND INTEGRATION GAPS

- [ ] **Connect Dynamic Categories in Wizard:** Fetch categories via `GET /categories` during Step 1 of Post a Job and remove the `seed-category-*` lookup map.
- [ ] **Align Notification Service Methods:** Update `notification.service.ts` to use `PATCH /notifications/:id/read` and `PUT /notifications/read`. Remove nonexistent preferences route.
- [ ] **Fix Worker Dashboard Header Counter:** Update `worker-dashboard-shell.tsx` so that `activeJobsCount` recalculates when `listWorkerJobs()` resolves.
- [ ] **Implement Backend Search Service:** Implement basic search queries in `search.service.ts` to support server-side job filtering.

### PHASE 3: MEDIUM-PRIORITY FALLBACKS & REAL-TIME UPGRADES

- [ ] **Upgrade Negotiation to WebSockets:** Wire `negotiation-modal.tsx` to real-time socket events instead of 5-second polling.
- [ ] **Eliminate Silent Mock Fallbacks:** Replace silent catch-block fallbacks in `earnings.service.ts` with user-facing error indicators and retry triggers.
- [ ] **Explicit GPS Permission States:** Replace silent Model Town coordinate fallbacks in `nearby-jobs-radar.tsx` with geolocation error states.

### PHASE 4: LOW-PRIORITY CLEANUP & STATIC CONTENT

- [ ] **Address Profile Defaults:** Replace default "Lahore" city fallback with explicit city selector.
- [ ] **Dynamic FAQ Management:** Optionally migrate static FAQs in `customer-help-view.tsx` to backend content endpoint.

## H Verification Checklist

During the subsequent fix phase, verify each item against running backend and frontend instances:

| COMPONENT / ENDPOINT | VERIFICATION TEST PROCEDURE | EXPECTED BEHAVIOR |
|---|---|---|
| `PUT /visits/:id/arrived` | Worker clicks "I Have Arrived" on active job card. Inspect network tab and server console. | Returns 200 OK; Redis stores OTP; Customer receives arrival socket event in real time. |
| `POST /payments` | Customer completes job and rates worker. Inspect database `Payment` and `WalletTransaction` tables. | Payment record created with status `COMPLETED`; worker earnings balance increments. |
| `GET /categories` | Open Post a Job Wizard Step 1. Observe category icons and category IDs submitted in Step 4 payload. | Payload contains valid DB UUIDs; job appears under correct category in worker feed. |
| `PATCH /notifications/:id/read` | Worker clicks notification in dropdown. Verify HTTP method in DevTools network tab. | Request succeeds with status 200; notification badge decrements immediately. |
| `Chat Voice Audio` | Record and send audio note. Play back from a mobile browser or secondary IP on local network. | Audio element loads source URL correctly without `localhost:4000` connection failure. |
| `POST /disputes` | Submit support ticket modal from customer portal. Verify database record. | Ticket saved in PostgreSQL; visible in admin dispute panel with unique DB ID. |